

Oracle SQL Subquery Logic Review & Execution Analysis

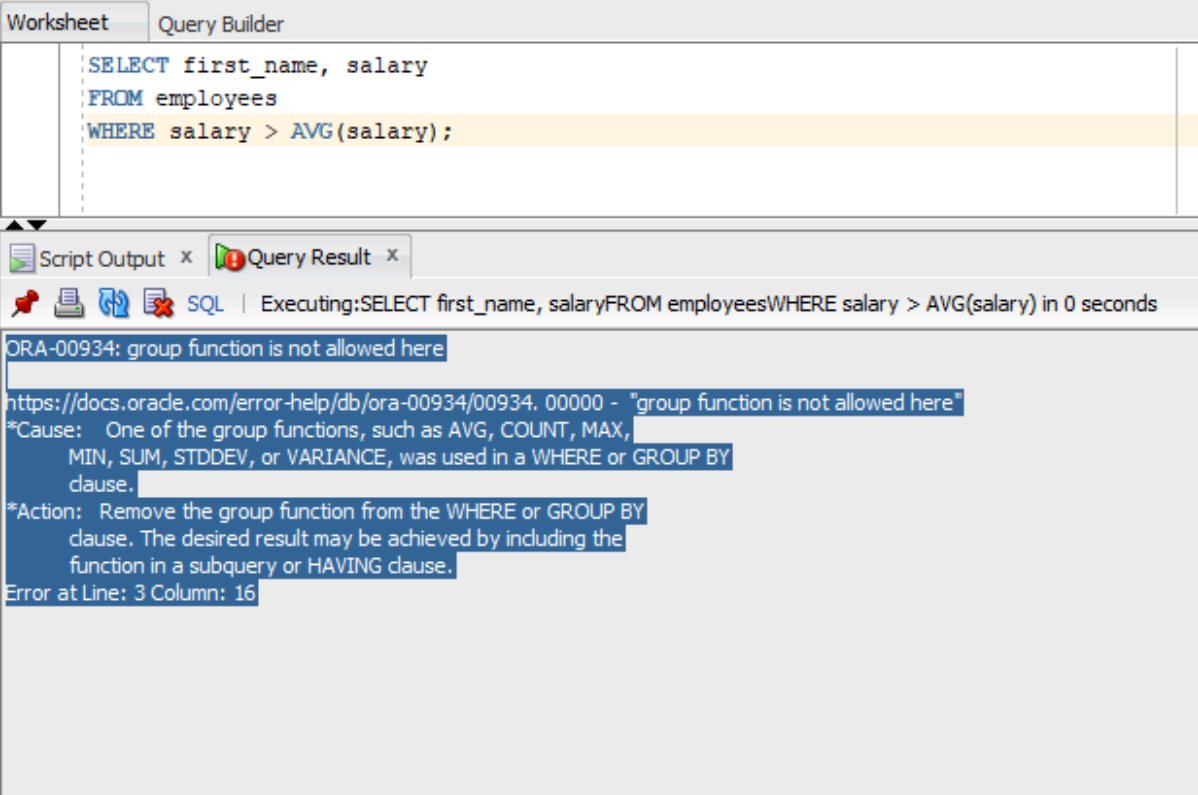
Query Review 1

Original Query:

```
SELECT first_name, salary
FROM employees
WHERE salary > AVG(salary);
```

Result:

ORA-00934: group function is not allowed here



The screenshot displays the Oracle SQL Developer interface. At the top, the 'Worksheet' tab is active, showing the following SQL query:

```
SELECT first_name, salary
FROM employees
WHERE salary > AVG(salary);
```

Below the query editor, the 'Script Output' and 'Query Result' tabs are visible. The 'Query Result' tab shows an error message:

ORA-00934: group function is not allowed here

The error details include a link to the Oracle documentation: [https://docs.oracle.com/error-help/db/ora-00934/00934.00000 - "group function is not allowed here"](https://docs.oracle.com/error-help/db/ora-00934/00934.00000 -). The cause is identified as: "One of the group functions, such as AVG, COUNT, MAX, MIN, SUM, STDDEV, or VARIANCE, was used in a WHERE or GROUP BY clause." The action recommended is: "Remove the group function from the WHERE or GROUP BY clause. The desired result may be achieved by including the function in a subquery or HAVING clause." The error is located at Line: 3 Column: 16.

Analysis:

The query is incorrect because `AVG(salary)` is an aggregate function, and aggregate functions cannot be used directly in the `WHERE` clause. The `WHERE` clause filters rows before the average salary is calculated.

Correct Query:

```

SELECT first_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);

```

Worksheet		Query Builder
		<pre> SELECT first_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); </pre>
Script Output x		Query Result x
		 Fetched 50 rows in 0.006 seconds
	FIRST_NAME	SALARY
1	Michael	13000
2	Susan	6500
3	Hermann	10000
4	Shelley	12008
5	William	8300
6	Steven	24000
7	Neena	17000
8	Lex	17000
9	Alexander	9000
10	Nancy	12008
11	Daniel	9000
12	John	8200
13	Ismael	7700
14	Jose Manuel	7800
15	Luis	6900
16	Den	11000
17	Matthew	8000
18	Adam	8200
19	Payam	7900
20	Shanta	6500
21	John	14000
22	Karen	13500
23	Alberto	12000
24	Gerald	11000
25	...	...

Explanation:

The subquery calculates the company average salary first. Then, the main query compares each employee's salary with that average. This is a single-row subquery because it returns only one value. The `>` operator is correct because the requirement is to find employees earning more than the average salary.

Conclusion:

The original logic was correct, but the syntax was wrong. Using a subquery fixes the error and correctly returns employees whose salary is above the company average.

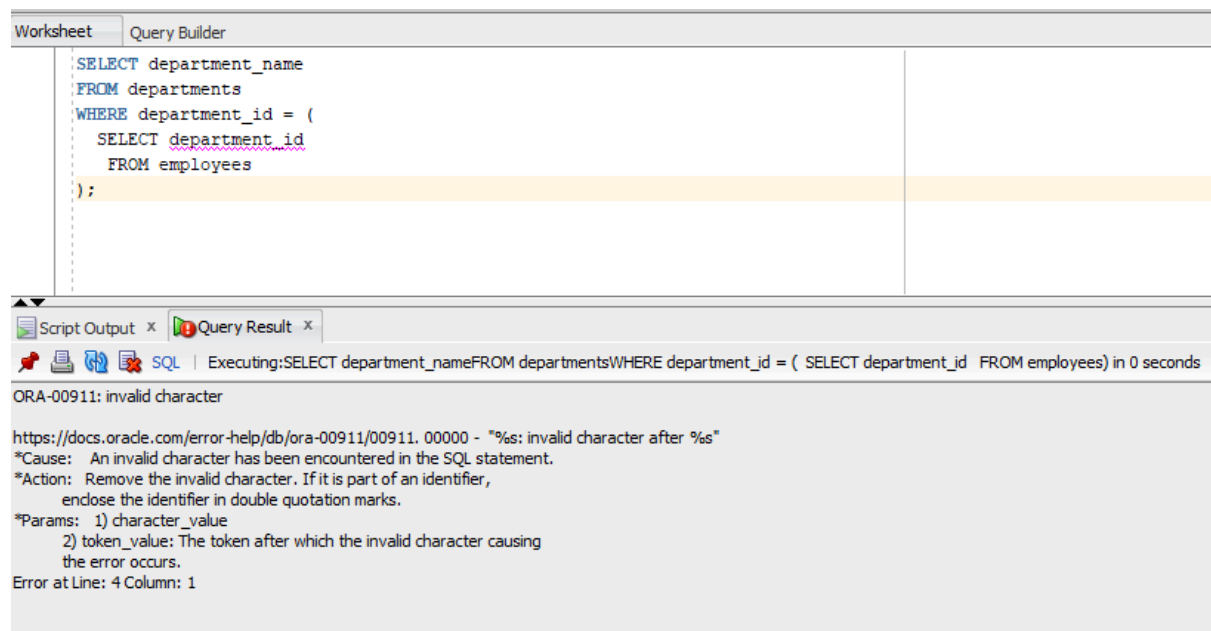
Query Review 2

Original Query:

```
SELECT department_name
FROM departments
WHERE department_id = (
    SELECT department_id
    FROM employees
);
```

Result:

ORA-00911: invalid character



The screenshot shows the SQL Developer interface. The top pane, titled 'Query Builder', contains the following SQL query:

```
SELECT department_name
FROM departments
WHERE department_id = (
    SELECT department_id
    FROM employees
);
```

The bottom pane, titled 'Script Output' and 'Query Result', displays the error message: 'ORA-00911: invalid character'. Below the error message, there is a link to the Oracle error documentation: [https://docs.oracle.com/error-help/db/ora-00911/00911.00000 - \"%s: invalid character after %s\"](https://docs.oracle.com/error-help/db/ora-00911/00911.00000 - \). The error details are as follows:

- *Cause: An invalid character has been encountered in the SQL statement.
- *Action: Remove the invalid character. If it is part of an identifier, enclose the identifier in double quotation marks.
- *Params: 1) character_value
- 2) token_value: The token after which the invalid character causing the error occurs.

Error at Line: 4 Column: 1

Analysis:

The error may happen because of an invalid hidden character, extra symbol, or copied formatting in the SQL statement. After removing the invalid character, the query still has a logic problem.

The subquery:

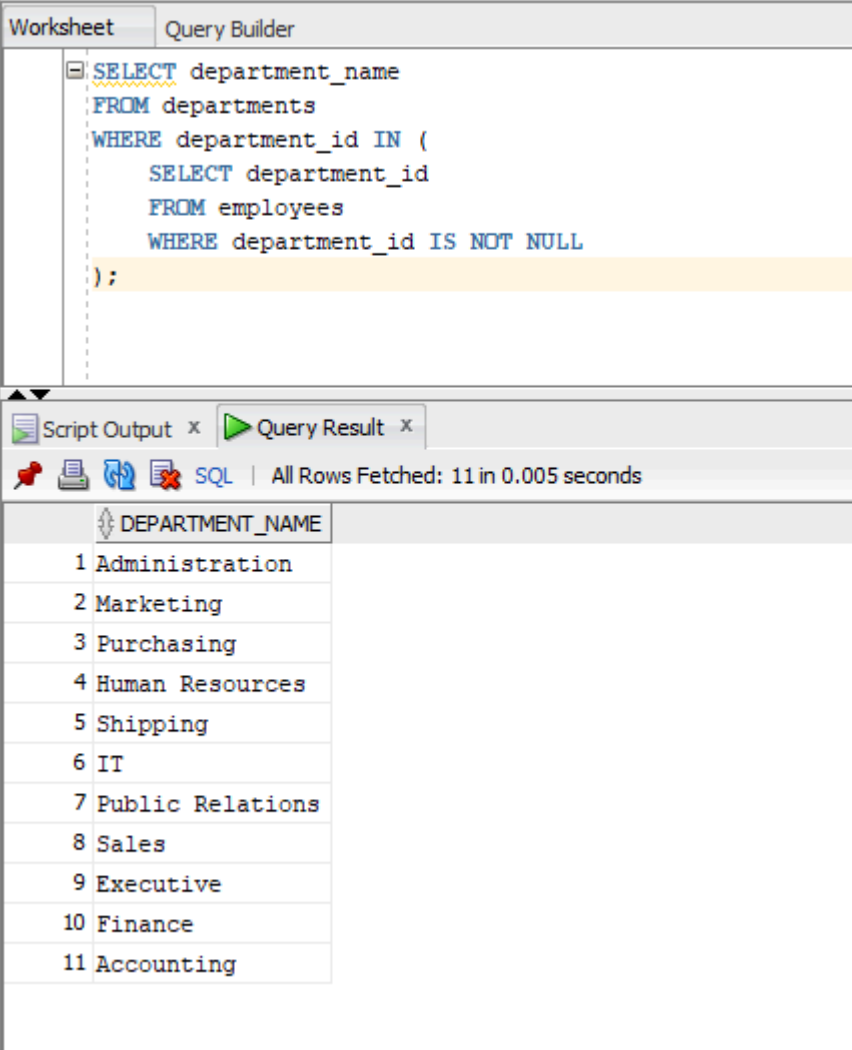
```
SELECT department_id  
FROM employees
```

returns many department IDs, but the main query uses `=`, which expects only one value.
This can cause another error:

ORA-01427: single-row subquery returns more than one row

Correct Query:

```
SELECT department_name  
FROM departments  
WHERE department_id IN (  
    SELECT department_id  
    FROM employees  
    WHERE department_id IS NOT NULL  
);
```



The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT department_name  
FROM departments  
WHERE department_id IN (  
    SELECT department_id  
    FROM employees  
    WHERE department_id IS NOT NULL  
);
```

Below the query editor, the 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'DEPARTMENT_NAME' and a row number. The table contains 11 rows of data.

	DEPARTMENT_NAME
1	Administration
2	Marketing
3	Purchasing
4	Human Resources
5	Shipping
6	IT
7	Public Relations
8	Sales
9	Executive
10	Finance
11	Accounting

Explanation:

The subquery returns multiple department IDs, so **IN** is the correct operator instead of **=**. This query shows departments that have employees assigned to them.

Conclusion:

The original query has a syntax issue and a logic issue. The improved query uses a multi-row subquery with **IN**, which correctly matches departments with employees.

Query Review 3

Original Query:

```
SELECT *  
FROM employees  
WHERE department_id IN (  
    SELECT salary  
    FROM employees  
);
```

Result:

No rows returned

The screenshot shows a SQL query editor with a 'Query Builder' tab. The query is as follows:

```
SELECT *  
FROM employees  
WHERE department_id IN (  
    SELECT salary  
    FROM employees  
);
```

Below the query editor, there is a 'Script Output' and 'Query Result' tab. The 'Query Result' tab shows 'All Rows Fetched: 0 in 0.001 seconds'. The result grid is empty, indicating no rows were returned.

EMPLOYEE...	FIRST_NA...	LAST_NAME	EMAIL	PHONE_N...	HIRE_DATE	JOB_ID	SALARY	COMMISS...	MANAGER...	DEPARTM...
-------------	-------------	-----------	-------	------------	-----------	--------	--------	------------	------------	------------

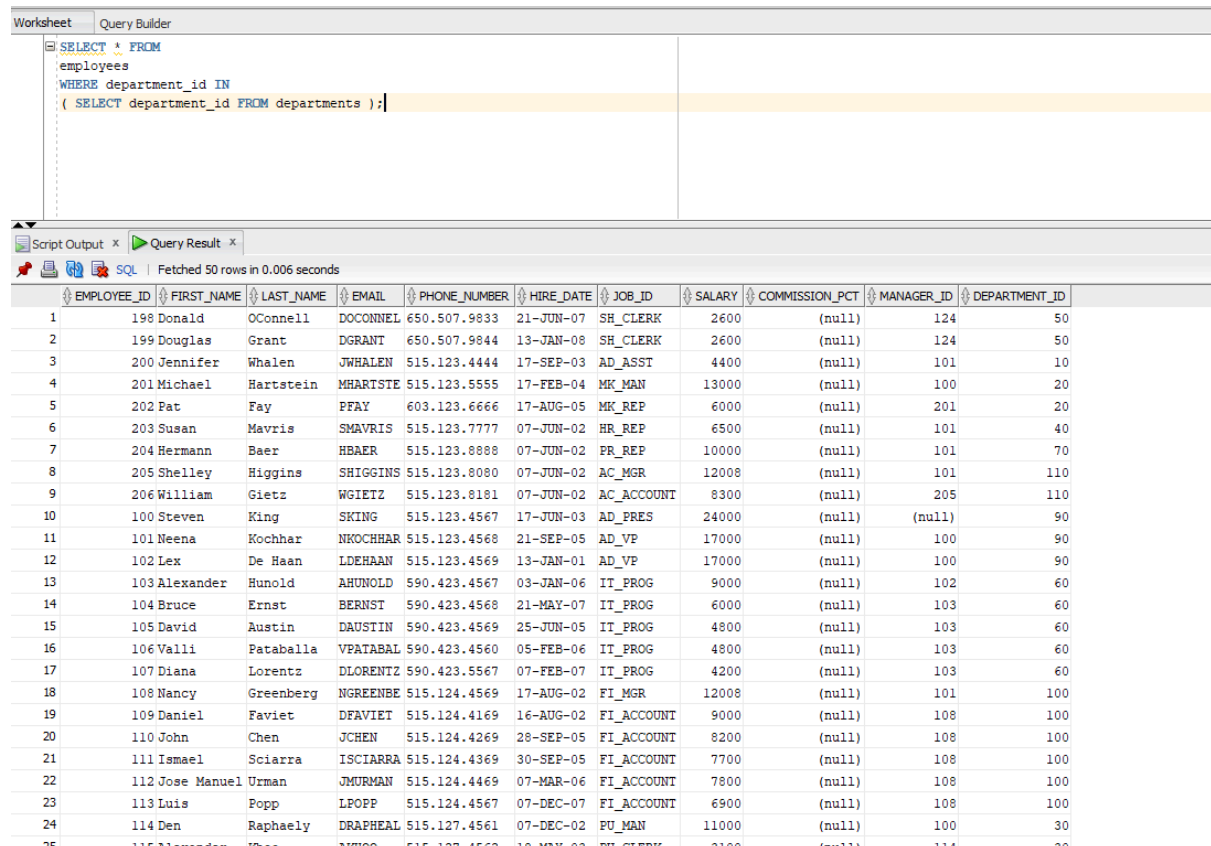
Analysis:

The query is syntactically correct, but the business logic is wrong. The main query compares **department_id** with **salary**, which are unrelated columns.

department_id contains department numbers, while **salary** contains employee salary values. Because these values do not match, the query returns empty rows.

Correct Query:

```
SELECT *  
FROM employees  
WHERE department_id IN (  
    SELECT department_id  
    FROM departments  
);
```



Worksheet Query Builder

```
SELECT * FROM  
employees  
WHERE department_id IN  
( SELECT department_id FROM departments );
```

Script Output x Query Result x

SQL | Fetched 50 rows in 0.006 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	198	Donald	OConnell	DOCONNEL	650.507.9833	21-JUN-07	SH_CLERK	2600	(null)	124	50
2	199	Douglas	Grant	DGRANT	650.507.9844	13-JAN-08	SH_CLERK	2600	(null)	124	50
3	200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-03	AD_ASST	4400	(null)	101	10
4	201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MAN	13000	(null)	100	20
5	202	Pat	Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000	(null)	201	20
6	203	Susan	Mavris	SMAVRIS	515.123.7777	07-JUN-02	HR_REP	6500	(null)	101	40
7	204	Hermann	Baer	HBAER	515.123.8888	07-JUN-02	PR_REP	10000	(null)	101	70
8	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-02	AC_MGR	12008	(null)	101	110
9	206	William	Gietz	WGIEZT	515.123.8181	07-JUN-02	AC_ACCOUNT	8300	(null)	205	110
10	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	(null)	(null)	90
11	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD_VP	17000	(null)	100	90
12	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-01	AD_VP	17000	(null)	100	90
13	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT_PROG	9000	(null)	102	60
14	104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-07	IT_PROG	6000	(null)	103	60
15	105	David	Austin	DAUSTIN	590.423.4569	25-JUN-05	IT_PROG	4800	(null)	103	60
16	106	Valli	Pataballa	VPATABAL	590.423.4560	05-FEB-06	IT_PROG	4800	(null)	103	60
17	107	Diana	Lorentz	DLORENTZ	590.423.5567	07-FEB-07	IT_PROG	4200	(null)	103	60
18	108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI_MGR	12008	(null)	101	100
19	109	Daniel	Faviet	DFAVIET	515.124.4169	16-AUG-02	FI_ACCOUNT	9000	(null)	108	100
20	110	John	Chen	JCHEN	515.124.4269	28-SEP-05	FI_ACCOUNT	8200	(null)	108	100
21	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30-SEP-05	FI_ACCOUNT	7700	(null)	108	100
22	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07-MAR-06	FI_ACCOUNT	7800	(null)	108	100
23	113	Luis	Popp	LPOPP	515.124.4567	07-DEC-07	FI_ACCOUNT	6900	(null)	108	100
24	114	Den	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	FU_MAN	11000	(null)	100	30
25	115	Alexander	Whalen	AWHALEN	515.123.4568	13-JAN-01	AD_VP	17000	(null)	100	90

Explanation:

The subquery should return department IDs, not salaries. This corrected query returns employees who belong to valid departments.

Conclusion:

The original query used the correct **IN** operator for a multi-row subquery, but it compared the wrong columns. The query logic must compare related values.

Query Review 4

Original Query:

```
SELECT first_name
```

```
FROM employees

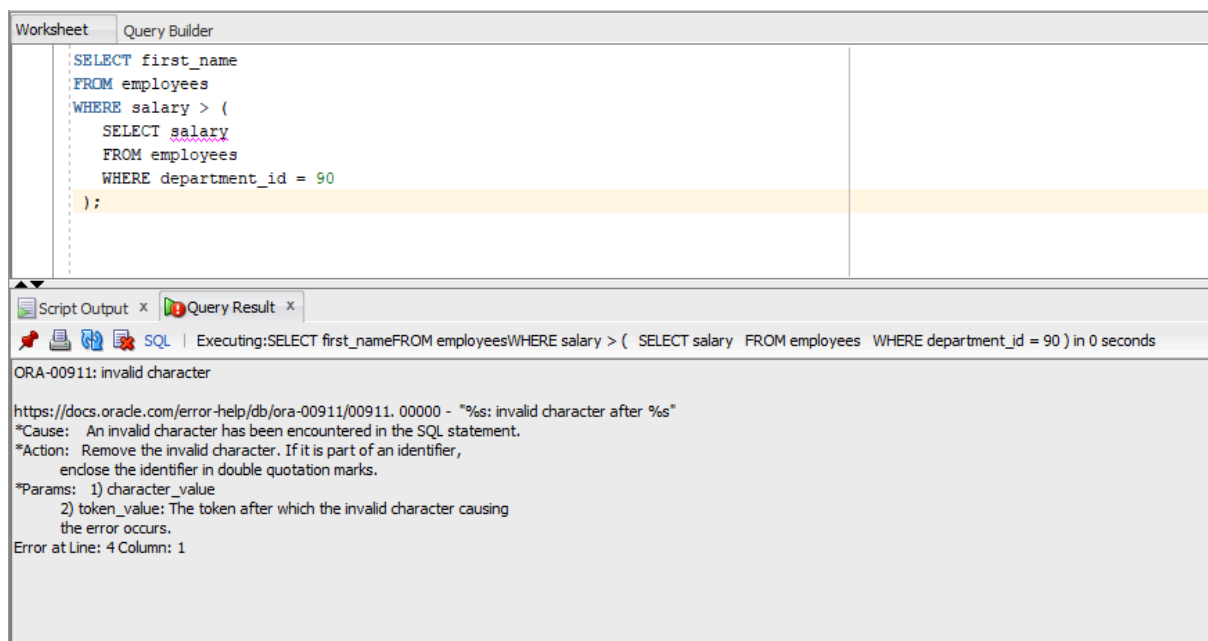
WHERE salary > (

    SELECT salary

    FROM employees

    WHERE department_id = 90

);
```



Result:

ORA-00911: invalid character

Analysis:

The error is likely caused by an invalid hidden character or copied formatting in the SQL code. After removing the invalid character, the query still has a logic problem.

The subquery returns multiple salaries from department 90, but the main query uses `>`, which expects only one value. This can cause:

ORA-01427: single-row subquery returns more than one row

Correct Query:

```
SELECT first_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = 90
);
```

The screenshot shows a SQL query builder window with two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT first_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = 90
);
```

Below the query editor, there is a 'Script Output' tab and a 'Query Result' tab. The 'Query Result' tab is active, showing the results of the query. The status bar indicates 'All Rows Fetched: 1 in 0.005 seconds'. The results are displayed in a table with two columns: 'FIRST_NAME' and 'SALARY'.

	FIRST_NAME	SALARY
1	Steven	24000

Explanation:

The corrected query compares each employee's salary with the average salary of department 90. The subquery returns one value, so using `>` is correct.

Conclusion:

The original query has a syntax issue and a subquery logic issue. A single-row comparison operator like `>` must be used with a subquery that returns only one value.